

Cahier des Charges

Système de Gestion des Étudiants

Application JavaFX avec MySQL

Architecture MVC + DAO

Lieu : Yaoundé, Cameroun

Date : Décembre 2025

Réalisé par : Ange Demanou et Delvira Yepmo

Table des matières

1	Présentation du Projet	3
1.1	Contexte	3
1.2	Objectif du Projet	3
1.3	Bénéfices Attendus	3
2	Spécifications Fonctionnelles	3
2.1	Gestion des Étudiants	3
2.1.1	Enregistrement d'un Nouvel Étudiant	3
2.1.2	Modification d'un Étudiant	4
2.1.3	Suppression d'un Étudiant	4
2.1.4	Affichage de la Liste des Étudiants	4
2.1.5	Recherche d'Étudiants	4
2.2	Gestion des Classes	4
2.2.1	Création d'une Classe	4
2.2.2	Opérations sur les Classes	5
2.3	Gestion des Inscriptions	5
2.3.1	Inscription d'un Étudiant dans une Classe	5
2.3.2	Consultation des Inscriptions	5
2.4	Filtres et Statistiques	5
3	Spécifications Techniques	5
3.1	Architecture Logicielle	5
3.2	Technologies Utilisées	6
3.3	Classes Minimales Attendues	6
3.3.1	Package Model	6
3.3.2	Package DAO	6
3.4	Modèle de Données	6
3.4.1	Table STUDENT	6
3.4.2	Table CLASSROOM	7
3.4.3	Table ENROLLMENT	7
4	Contraintes et Règles Métier	7
4.1	Contraintes d'Intégrité	7
5	Interface Utilisateur	7
5.1	Écran Principal	7
5.2	Formulaires	8
5.2.1	Formulaire d'Ajout/Modification	8
5.3	Ergonomie	8
6	Livrables	8
6.1	Documentation	8
6.2	Code Source	9
6.3	Application	9
7	Planification	9

8 Critères d'Acceptation	9
9 Conclusion	9

1 Présentation du Projet

1.1 Contexte

Un établissement d'enseignement supérieur souhaite informatiser la gestion de ses étudiants. Actuellement, la gestion est effectuée de manière manuelle ce qui entraîne :

- Des pertes de temps dans la recherche d'informations
- Des risques d'erreurs et de doublons
- Une difficulté à générer des statistiques
- Un manque de traçabilité des inscriptions

1.2 Objectif du Projet

Développer un **système de gestion des étudiants** permettant au personnel administratif de :

- Enregistrer de nouveaux étudiants
- Modifier les informations des étudiants existants
- Supprimer des étudiants de la base
- Consulter et rechercher des étudiants
- Gérer les classes et les inscriptions
- Garantir l'unicité des matricules

1.3 Bénéfices Attendus

- **Gain de temps** : Automatisation des tâches répétitives
- **Fiabilité** : Réduction des erreurs de saisie
- **Accessibilité** : Recherche rapide et efficace
- **Sécurité** : Protection des données sensibles

2 Spécifications Fonctionnelles

2.1 Gestion des Étudiants

2.1.1 Enregistrement d'un Nouvel Étudiant

Le personnel doit pouvoir enregistrer un étudiant en fournissant :

Règles de validation :

- Le matricule doit être unique (le système refuse les doublons)
- L'email doit respecter le format standard (exemple@domaine.cm)
- La date de naissance doit être cohérente (âge > 15 ans)
- Tous les champs obligatoires doivent être remplis

Champ	Description	Obligatoire
Id	Identifiant unique (ex : 24H2005)	Oui
Nom	Nom de famille	Oui
Prénom	Prénom(s)	Oui
Date de naissance	Format : JJ/MM/AAAA	Oui
Email	Adresse email valide	Non
Sexe	Masculin (M) ou Féminin (F)	Oui
Classe	Classe d'inscription	Oui

TABLE 1 – Champs de saisie pour un étudiant

2.1.2 Modification d'un Étudiant

Le personnel peut modifier toutes les informations d'un étudiant existant, à l'exception de son ID unique et generer automatiquement en base de données.

2.1.3 Suppression d'un Étudiant

La suppression d'un étudiant entraîne :

- La suppression de l'étudiant de la base de données
- La suppression de toutes son inscription
- Une demande de confirmation avant l'action

2.1.4 Affichage de la Liste des Étudiants

Le système affiche tous les étudiants dans un tableau avec les colonnes suivantes :

- ID
- Matricule
- Nom
- Prénom
- Date de naissance
- Email
- Sexe

2.1.5 Recherche d'Étudiants

Le personnel peut rechercher un étudiant par :

- **Nom** : Recherche partielle (ex : "Dem" trouve "Demanou")
- **Matricule** : Recherche exacte ou partielle

La recherche s'effectue en temps réel lors de la saisie dans le champ de recherche.

2.2 Gestion des Classes

2.2.1 Création d'une Classe

Le personnel peut créer une nouvelle classe en spécifiant :

- **Nom** : Ex : "Licence 1 Informatique"
- **Niveau** : Ex : L1, L2, L3, M1, M2

2.2.2 Opérations sur les Classes

- Modifier le nom ou le niveau d'une classe
- Supprimer une classe (si aucun étudiant n'y est inscrit)
- Lister toutes les classes disponibles
- Consulter les étudiants d'une classe

2.3 Gestion des Inscriptions

2.3.1 Inscription d'un Étudiant dans une Classe

Le système permet d'inscrire un étudiant dans une ou plusieurs classes.

Contraintes :

- Un étudiant ne peut être inscrit qu'une seule fois dans la même classe pour une année donnée
- La date d'inscription est enregistrée automatiquement

2.3.2 Consultation des Inscriptions

Le personnel peut consulter :

- Toutes les inscriptions d'un étudiant
- Tous les étudiants d'une classe

2.4 Filtres et Statistiques

- **Filtrer par classe** : Afficher uniquement les étudiants d'une classe
- **Filtrer par niveau** : Afficher les étudiants par niveau (L1, L2, etc.)
- **Statistiques** : Nombre total d'étudiants, répartition par sexe, par classe

3 Spécifications Techniques

3.1 Architecture Logicielle

Le système utilise l'architecture **MVC (Modèle-Vue-Contrôleur)** combinée au pattern **DAO (Data Access Object)**.

Couche	Responsabilité
Modèle	Classes représentant les entités métier (Student, Classroom, Enrollment)
Vue	Interface graphique JavaFX (fichiers FXML)
Contrôleur	Logique de présentation et gestion des événements
DAO	Accès et manipulation des données en base

Technologie	Version	Usage
Java	11+	Langage de programmation
JavaFX	21	Interface graphique utilisateur
MySQL	-	Système de gestion de base de données
Maven	3.8+	Gestion des dépendances
WAMP/XAMPP	-	Serveur local (développement)

3.2 Technologies Utilisées

3.3 Classes Minimales Attendues

3.3.1 Package Model

- **Student** : Représente un étudiant
 - Attributs : id, matricule, nom, prenom, dateNaissance, email, sexe
 - Méthodes : getters, setters, isValidEmail()
- **ClassRoom** : Représente une classe
 - Attributs : id, nom, niveau
 - Méthodes : getters, setters
- **Enrollment** : Représente une inscription
 - Attributs : id, student, classRoom, year, dateInscription
 - Méthodes : getters, setters

3.3.2 Package DAO

- **DatabaseConnection** : Gère la connexion MySQL (Singleton)
- **StudentDAO** : CRUD complet pour les étudiants
 - addStudent(Student) : boolean
 - getAllStudents() : List<Student>
 - updateStudent(Student) : boolean
 - deleteStudent(int id) : boolean
 - searchStudents(String keyword) : List<Student>
 - matriculeExists(String matricule) : boolean
- **ClassRoomDAO** : CRUD pour les classes
- **EnrollmentDAO** : Gestion des inscriptions

3.4 Modèle de Données

3.4.1 Table STUDENT

```
CREATE TABLE student (
  id INT AUTO_INCREMENT PRIMARY KEY,
  matricule VARCHAR(20) UNIQUE NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  date_naissance DATE NOT NULL,
```

```
email VARCHAR(150),  
sexe ENUM('M', 'F') NOT NULL,  
);
```

3.4.2 Table CLASSROOM

```
CREATE TABLE classroom (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(100) NOT NULL,  
    niveau VARCHAR(50) NOT NULL,  
);
```

3.4.3 Table ENROLLMENT

```
CREATE TABLE enrollment (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    id_student INT NOT NULL,  
    id_classroom INT NOT NULL,  
    year INT NOT NULL,  
    date_inscription DATE,  
    FOREIGN KEY (id_student) REFERENCES student(id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (id_classroom) REFERENCES classroom(id)  
        ON DELETE CASCADE,  
    UNIQUE KEY (id_student, id_classroom, year)  
);
```

4 Contraintes et Règles Métier

4.1 Contraintes d'Intégrité

1. **Unicité du matricule** : Deux étudiants ne peuvent pas avoir le même matricule
2. **Format email** : L'email doit respecter le format standard
3. **Intégrité référentielle** :
 - Une inscription doit référencer un étudiant existant
 - Une inscription doit référencer une classe existante
4. **Suppression** : La suppression d'un étudiant supprime ses inscriptions
5. **Unicité de l'inscription** : Un étudiant ne peut s'inscrire qu'une fois par classe et par année

5 Interface Utilisateur

5.1 Écran Principal

L'écran principal comprend :

- **En-tête** : Titre de l'application et logo de l'établissement si possible

- **Barre de recherche** : Champ de saisie pour rechercher un étudiant
- **Tableau** : Liste de tous les étudiants avec colonnes
- **Boutons d'action** :
 - Ajouter
 - Modifier
 - Supprimer
- **Filtres** : Box pour filtrer par classe
- **Barre de statut** : Messages d'information en bas de l'écran

5.2 Formulaire

5.2.1 Formulaire d'Ajout/Modification

Le formulaire contient :

- Champs de saisie pour tous les attributs
- Labels explicites
- Validation en temps réel
- Messages d'erreur clairs
- Boutons "Enregistrer" et "Annuler"

5.3 Ergonomie

- Interface intuitive et responsive
- Navigation claire entre les écrans
- Confirmation avant toute suppression
- Messages de succès/erreur visuels
- Raccourcis clavier (Enter pour valider)

6 Livrables

6.1 Documentation

1. **Cahier des charges** (ce document)
2. **Diagrammes UML** :
 - Diagramme de cas d'usage
 - Diagramme de classes
3. **Modèle de données** :
 - MCD (Modèle Conceptuel de Données)
 - MLD (Modèle Logique de Données)

6.2 Code Source

- Code source Java complet et commenté
- Fichiers FXML pour l'interface
- Scripts SQL de création de la base
- Fichier pom.xml (configuration Maven)
- Fichier README avec instructions d'installation

6.3 Application

- Application JavaFX fonctionnelle
- Base de données MySQL avec données de test

7 Planification

Phase	Tâches	Durée
Analyse	Rédaction cahier des charges, diagrammes UML	- jours
Conception	Modèle de données, architecture logicielle	- jours
Développement	Codage des classes, interface JavaFX	- jours
Tests	Tests unitaires, tests d'intégration	- jours
Documentation	Rédaction manuels, commentaires code	- jours
Déploiement	Installation, formation utilisateurs	- jour
Total		– jours

8 Critères d'Acceptation

Le projet sera considéré comme accepté si :

1. **Fonctionnalités** : Toutes les fonctionnalités décrites seront implémentées et opérationnelles
2. **Performance** : Le système répond en moins de 5 secondes pour toute opération
3. **Fiabilité** : Aucune perte de données lors des opérations CRUD
4. **Sécurité** : Les contraintes d'intégrité sont respectées (matricule unique, etc.)
5. **Ergonomie** : L'interface est intuitive et facile à utiliser
6. **Documentation** : Toute la documentation est complète et à jour
7. **Tests** : Tous les tests unitaires et d'intégration seront effectués

9 Conclusion

Ce cahier des charges définit l'ensemble des spécifications fonctionnelles et techniques pour le développement d'un système de gestion des étudiants.

Le respect de ces spécifications garantira la livraison d'une application robuste, fiable et facile à utiliser, répondant parfaitement aux besoins de l'utilisateur.

Contact :

Lieu : Yaoundé, Cameroun

Email : angedemanou0@gmail.com et dedeshake5@gmail.com

Date : Décembre 2025